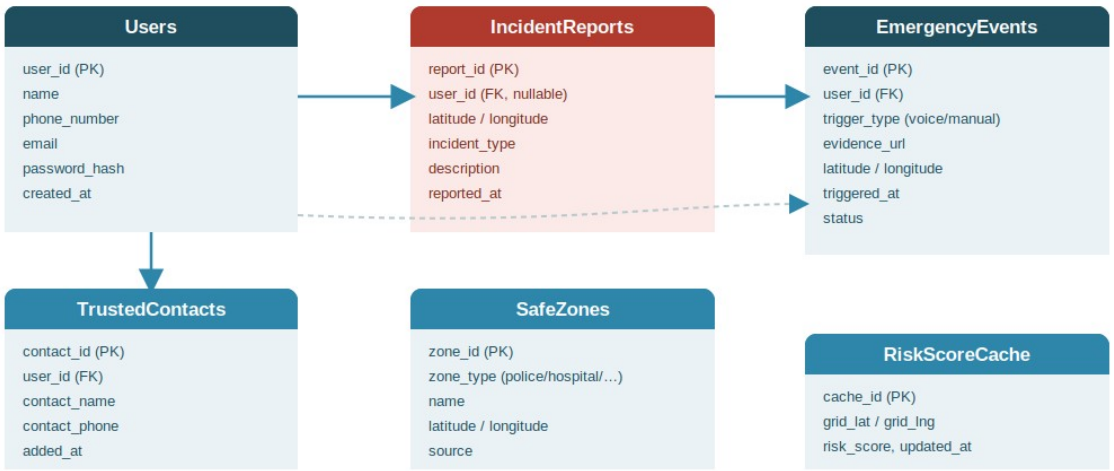
SAFEPATH

Technology Setup & Database Design

1. Database Schema

The core database structure was designed to support all planned features — users, trusted contacts, incident reports, safe zones, and emergency events:



PK = Primary Key FK = Foreign Key Dashed line = feeds risk-prediction model

2. Backend Project Structure

The backend project (Node.js / FastAPI) was initialized with the following structure:

```
backend/
  ├── routes/          → auth, contacts, incidents, risk-score, emergency
  ├── controllers/     → request handling logic for each route
  ├── models/          → Users, TrustedContacts, IncidentReports, SafeZones
  ├── config/          → database connection, environment variables
  └── server.js        → application entry point
```

3. Initial API Endpoints Planned

The following core APIs were defined to support the app's first features:

Method	Endpoint	Purpose
POST	/api/auth/signup	Register a new user
POST	/api/auth/login	Authenticate user, return token
POST	/api/contacts	Add a Trusted Contact
GET	/api/contacts	List a user's Trusted Contacts
GET	/api/risk-score?lat&lng	Get route/area risk score

Method	Endpoint	Purpose
GET	/api/safe-zones?lat&lng	Get nearby safe zones
POST	/api/incidents	Submit a community incident report
POST	/api/emergency/trigger	Start an emergency event (voice/manual)

4. Crime & Safety Data Sources Identified

Data sources were identified for each type of information the risk-prediction model will need:

Data Type	Source
Crime Records	NCRB (National Crime Records Bureau) open data; state police open-data portals
Crowd Density	In-app user activity + public transit/foot-traffic estimates
Weather & Time	Public weather API + device system time
Community Reports	Crowdsourced, in-app user-submitted incident reports